МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО МГТУ «СТАНКИН»)

АРХИТЕКТУРНОЕ МОДЕЛИРОВАНИЕ ПРОГРАММНЫХ СИСТЕМ НА ОСНОВЕ МОДЕЛИ C4

Методическое пособие к лабораторным работам

Москва, 2026

Введение

Настоящее учебно-методическое пособие предназначено для получения практических навыков проектирования сложных программных систем с использованием современной методологии архитектурной визуализации C4 Model.

Модель C4, предложенная Саймоном Брауном, эффективно решает проблему абстракции и запутанности классических UML-диаграмм, предлагая иерархический подход к документированию архитектуры, аналогичный масштабированию географических карт.

Пособие включает в себя 5 лабораторных работ. Необходимо последовательно раскрыть уровни абстракции C4 Model от макроуровня системного ландшафта и контекста до программного кода и рантайм-динамики процессов применительно к индивидуальным темам курсовых проектов. Описанный далее кейс «Умная парковка» (Smart Parking) служит исключительно сквозным демонстрационным примером и не подлежит выполнению.

Системные архитекторы, разработчики и аналитики в современных Agile-командах отказываются от UML в пользу модели C4, потому что классический UML перестал справляться с темпом разработки. Вместо того чтобы помогать команде, перегруженный правилами UML заставляет людей часами спорить о правильности стрелочек и типов связей, создавая сложные схемы, которые никто не читает и которые устаревают в тот же день.

Модель C4 решает эту проблему: она убирает лишний академизм и предлагает простую, интуитивно понятную структуру «зумирования» проекта, как в Google Картах. Это позволяет быстро объяснить устройство системы и бизнесу, и разработчикам, не утопая в бюрократии.

Создатель модели C4, британский архитектор Саймон Браун, сформулировал эту проблему очень похоже: он утверждал, что после заката эпохи «тяжелого» UML команды погрязли в хаосе из непонятных стрелочек и квадратиков на маркерных досках, потеряв способность связно объяснять архитектуру.

Кто переходит на модель C4?

- Software-архитекторы и Lead-разработчики — которым нужно быстро и безболезненно вводить новых программистов в проект.
- Системные аналитики — которым необходимо согласовывать технические решения и с разработкой, и с бизнесом.
- Agile-команды (Scrum/Kanban) — работающие в условиях частых изменений, где классическая тяжеловесная документация UML устаревает быстрее, чем ее успевают дорисовать.

Почему переходят на C4? (Решение проблем UML).

Модель C4 — это не замена UML, а подход «сначала абстракция». Она решает проблему «связывания в единую историю» за счет нескольких ключевых факторов:

1. Принцип «Google Карт» - четкая стартовая точка и зумирование.

Модель C4 предоставляет строго 4 уровня абстракции:

- L1: Context (Контекст) — Максимальный масштаб. Показывает вашу систему как единое целое, ее пользователей и связь с другими системами. Идеально для бизнеса.
- L2: Container (Контейнеры) — Зум на один шаг. Показывает отдельно развернутые сущности: мобильное приложение, фронтенд, бэкенд-API, базы данных.
- L3: Component (Компоненты) — Зум внутрь контейнера. Показывает внутренние модули конкретного бэкенда (например, модуль авторизации, платежный шлюз).
- L4: Code (Код) — Максимальное приближение. Схемы классов или сущностей. В C4 этот уровень часто опускают, так как код меняется слишком быстро. История рассказывается последовательно: от общего ландшафта (L1) до конкретных модулей (L3

2. Фокус на архитектурных границах, а не на синтаксисе.

UML заставляет думать о правильности стрелочек (ассоциация, агрегация, композиция, зависимость). C4 предлагает сфокусироваться на сути: «Что это за штука, на каких технологиях работает и как именно общается с соседней штукой

3. Понятный язык для разной аудитории.

Уровни C4 идеально делят «историю» под конкретного слушателя:

- Стейкхолдерам и продакт-менеджерам показывают уровень Context.
- DevOps-инженерам и смежным командам интеграции — уровень Container.
- Разработчикам внутри команды — уровень Component.

4. Идеальный союз с концепцией Diagrams-as-Code.

C4 великолепно ложится на текстовое описание диаграмм (например, через инструмент C4-PlantUML или Structurizr).

Архитектуру можно хранить прямо в репозитории с кодом в виде текста, версионировать через Git и автоматически перерисовывать при изменениях.

Переход на C4 происходит потому, что C4 дает готовую структуру повествования. UML предоставляет огромную «коробку с деталями LEGO», но не инструкцию по сборке.

C4 — это готовый шаблон, который позволяет не утонуть в деталях на ранних этапах проектирования.

Несмотря на то, что C4 Model убирает академическую перегруженность UML, она не исключает его полностью, а гармонично дополняет на нижних уровнях абстракции. Модель C4 великолепно описывает статическую структуру (из каких «кубиков» состоит система), но когда команде необходимо спроектировать сложный динамический процесс или детально проработать внутренние контракты, архитекторы точечно используют элементы UML.

Это нужно для того, чтобы не изобретать велосипед и задействовать строгий, общепринятый язык там, где C4 намеренно сохраняет верхнеуровневую абстракцию. В контексте системы «Умная парковка» такое гибридное использование раскрывается на двух ключевых уровнях:

- Диаграмма компонентов (UML Component Diagram) на уровне L3. Когда мы делаем «зум» внутрь контейнера бэкенда (Parking Backend API), C4-диаграмма компонентов показывает модули системы: ReservationComponent, BillingComponent и NotificationComponent. Чтобы жестко зафиксировать интерфейсы взаимодействия между ними в коде на Python, C4-схема может быть расширена элементами диаграммы компонентов UML — в частности, явным выделением предоставляемых и требуемых портов (интерфейсов) для интеграции со сторонними платежными шлюзами или IoT-датчиками парковочных мест.
- Диаграмма классов (UML Class Diagram) на уровне L4. Уровень кода в C4 Model практически полностью отдается на откуп диаграммам классов UML. Для системы на Python это критически важно при проектировании объектно-ориентированной модели (Domain Model). Например, с помощью UML-диаграммы классов визуализируется внутренняя логика компонента бронирования: связи между классами ParkingSlot (парковочное место), Reservation (бронь), User (водитель) и Tariff (тарифная сетка), с указанием их атрибутов, методов и типов отношений (композиция, наследование).
- Диаграмма последовательности (UML Sequence Diagram) как сквозной процесс. Сама по себе модель C4 статична. Чтобы показать, как именно компоненты взаимодействуют во времени, используется диаграмма последовательности UML. Например, для сценария «Бронирование парковочного места пользователем через мобильное приложение» UML-диаграмма последовательности пошагово визуализирует таймлайн: как запрос от Mobile App проходит через API-шлюз, как компонент бронирования запрашивает статус датчика у IoT-модуля, резервирует место в БД и отправляет пуш-уведомление водителю.

1 ИНДИВИДУАЛЬНЫЕ ЗАДАНИЯ И ПРАВИЛА ВЫПОЛНЕНИЯ

1.1 Лабораторная работа 1. C4 Model.

В соответствии с определенной для курсового проекта темой исследования:

1.1.1 Диаграмма системного контекста

- Разработать **диаграмму системного контекста - System Context Diagram.**
- Оформить отчет по лабораторной работе и разместить его в электронной образовательной среде.

Требования к разработке диаграммы:

- Очертить жесткие внешние границы проектируемой системы, представив ее в виде единого неделимого «черного ящика».
- Диаграмма должна содержать центральный блок и не менее 3 различных пользовательских ролей (персон).
- Диаграмма должна включать не менее 3 внешних программных или аппаратных систем, находящихся вне зоны прямого контроля разработчиков.
- Все связи между элементами должны быть строго направленными с обязательным текстовым описанием передаваемых данных.

1.1.2 Диаграмма контейнеров

- Разработать **диаграмму контейнеров (Container Diagram).**
- Сформировать подробную таблицу описания каждого контейнера.
- Оформить отчет по лабораторной работе и разместить его в электронной образовательной среде.

Требования к разработке диаграммы:

- Осуществить декомпозицию центрального системного блока на обособленные, независимо развертываемые исполняемые сущности (контейнеры).
- Выделить не менее 5 технологических контейнеров (включая мобильное и веб-приложение, бэкенд-сервисы и хранилища данных).
- Для каждого контейнера в обязательном порядке зафиксировать конкретный технологический стек (языки, фреймворки, СУБД).
- Для всех межсервисных связей четко указать используемые сетевые протоколы взаимодействия.

1.2 Лабораторная работа 2. C4 Model.

В соответствии с темой курсовой работы:

1.2.1 Диаграмма компонентов

- Разработать **диаграмму компонентов (Component Diagram)** для ключевого контейнера.
- Подготовить текстовое обоснование и спецификацию внутренних интерфейсов.
- Оформить отчет по лабораторной работе и разместить его в электронной образовательной среде.

Требования к разработке диаграммы:

- В соответствии с индивидуальной темой курсового проекта: выполнить декомпозицию ключевого контейнера бизнес-логики (например, бэкенда) на внутренние логические компоненты.
- Диаграмма должна содержать не менее 4 внутренних взаимосвязанных компонентов, выполняющихся в едином адресном пространстве.

- Продемонстрировать явное разделение ответственности между компонентами в соответствии с принципами многослойной архитектуры.

1.3 Лабораторная работа 3. C4 Model.

В соответствии с темой курсовой работы:

1.3.1 Диаграмма кода

- В соответствии с индивидуальной темой курсового проекта: Разработать диаграмму классов UML (UML Class Diagram) для выбранного компонента и реализовать его прототип на языке программирования Python (или стеке проекта).
- Реализовать чистый объектно-ориентированный код на языке Python.
- Оформить отчет по лабораторной работе и разместить его в электронной образовательной среде.

Требования к разработке диаграммы и кода:

- Отобразить объектно-ориентированную структуру конкретного архитектурного компонента на уровне классов, интерфейсов и типов связей.
- Обязательное использование шаблона проектирования «Стратегия» (Strategy) для динамической подмены алгоритмов бизнес-логики.
- Программный код должен быть снабжен строгой аннотацией типов (Type Hinting) и соответствовать принципам SOLID.

1.4 Лабораторная работа 4. C4 Model.

1.4.1 Макроархитектура, динамические процессы и инфраструктура развертывания.

В соответствии с индивидуальной темой курсовой работы:

- Разработать **диаграмму системного ландшафта (Level 0),** отображающую интеграцию продукта в корпоративную среду.
- Разработать **динамическую диаграмму (Dynamic Diagram)** для сквозного процесса взаимодействия.
- Зафиксировать пошаговые сценарии и протоколы обмена сообщениями в режиме реального времени.
- Разработать Диаграмму развертывания для уровня контейнеров (Level 2: Container Diagram).
- Оформить отчет по лабораторной работе и разместить его в электронной образовательной среде.

Требования к разработке Системного ландшафта и Динамической диаграмм:

- На уровне ландшафта (Level 0) показать встраивание системы в ИТ-инфраструктуру предметной области вашего проекта. с выделением внешних ИТ-границ.
- Отразить взаимодействие целевой системы как минимум с 2 смежными городскими/корпоративными системами (например, CRM, IAM) и 1 внешней финансовой системой с обязательным указанием протоколов (gRPC, HTTPS, OAuth2).
- На уровне динамики построить модель сквозного процесса (например, «Регистрация изменения статуса парковочного места при заезде автомобиля»).
- Динамическая диаграмма должна наглядно демонстрировать хронологический порядок вызовов с использованием последовательной нумерации.

Требования к разработке Диаграммы развертывания (Deployment Diagram):

1. Отобразить физическую или виртуальную инфраструктуру (наземные сервера, облачные провайдеры, IoT-периферия), на которую разворачиваются технологические контейнеры системы.

2. Выделить логические узлы (Nodes) и контейнеры сред выполнения (Docker-контейнеры, базы данных, веб-сервера, мобильные ОС).

3. Указать конкретные параметры сетевой безопасности и взаимодействия узлов (порты, шифрование TLS, протоколы передачи данных MQTTS, HTTPS, TCP).

4. Отразить топологию распределения и отказоустойчивости (например, репликацию БД Master-Replica, балансировку нагрузки через Nginx Reverse Proxy).

1.5 Лабораторная работа 5. C4 Model. Интеграция, сборка проекта.

Пятый этап проектирования завершает цикл разработки архитектурного описания программной системы. В рамках этой лабораторной работы статическая и динамическая модели C4, созданные на предыдущих этапах, интегрируются в единый Git-репозиторий проекта. Основная цель работы — практическое освоение концепции Diagrams-as-Code (диаграммы как код), развертывание пайплайна непрерывной интеграции (CI/CD) для автоматической валидации синтаксиса и генерации актуальной графической документации проекта.

В соответствии с темой курсовой работы:

- Провести интеграцию всех слоев архитектурного описания в единый репозиторий, реализация принципа Diagrams-as-Code и внедрение пайплайнов автоматической валидации и отрисовки схем.

Требования к выполнению и защите работы:

1. Тимлид настраивает единый репозиторий проекта со строгой модульной структурой и подключает CI/CD автоматизацию.
2. Разработчики интегрируют свои локальные puml-файлы в общую ветку через механизм Pull Requests, проходя процедуру Code Review со стороны Тимлида.
3. Сборка проекта в CI/CD должна автоматически валидировать синтаксис PlantUML и

генерировать растровые схемы (PNG/SVG) для базы знаний проекта. При наличии синтаксических ошибок сборка должна падать с индикатором 'Красный крест' (✕).
4. Предоставить логи сборки и сгенерированные артефакты для индивидуальной защиты работы преподавателю.

2 ПОСТАНОВКА ЗАДАЧИ. ОПИСАНИЕ ПРИМЕРА

Описание сквозного проекта: Система «Умная парковка»

Прежде чем приступать к архитектурному моделированию, необходимо четко определить целевую систему, ее границы и бизнес-контекст. В качестве примера единой сквозной предметной области в рамках данного курса рассматривается программная система «Умная парковка», реализуемая с применением современного стека технологий на языке программирования Python. Диаграммы необходимо строить строго по собственной теме курсовой работы.

Smart Parking System Целевая система:

— это комплексное высокотехнологичное программное обеспечение, предназначенное для автоматизации процессов поиска, бронирования, краткосрочной/долгосрочной аренды и оплаты парковочных мест в режиме реального времени на основе данных от IoT-датчиков и интеграции со смежными городскими службами.

Служебное назначение системы заключается в решении следующих ключевых задач:

1. **Для водителей:** Оптимизация городского трафика за счет сокращения времени, затрачиваемого водителями на поиск свободных мест.
2. **Для операторов парковок:** Повышение собираемости платежей, прозрачности распределения парковочного пространства, сокращение операционных издержек и предоставление гибких аналитических инструментов.
3. **Для муниципальных органов:** Сбор детальной аналитики загруженности дорожной сети для планирования градостроительной и транспортной политики.

Базовая бизнес-логика системы опирается на непрерывный сбор телеметрии с аппаратных IoT-датчиков занятости мест. Система обрабатывает входящие события, обновляет глобальную карту доступности парковочных зон, рассчитывает стоимость сессий по динамическим алгоритмам и проводит транзакции через внешний эквайринг. Архитектура системы должна обеспечивать высокую отказоустойчивость, низкую задержку обработки данных и четкое разграничение ответственности между микросервисами.

3 РЕКОМЕНДАЦИИ ПО ИСПОЛЬЗОВАНИЮ ПРОГРАММНЫХ ИНСТРУМЕНТОВ

3.1 Инструмент PlantUML и библиотека C4-PlantUML

В рамках данного курса в качестве базового инструмента проектирования используется PlantUML. Реализация методологии C4 основана на подключении специализированного open-source репозитория макросов с помощью директивы !include в начале каждого файла. Это

позволяет использовать нативные абстракции модели — Person, Container, Component вместо базовых графических примитивов.

Для построения читаемых схем крайне важно управлять структурой элементов с помощью корректных связей. Для обеспечения стабильной автоматической сборки проекта через CI/CD-пайплайны и исключения конфликтов визуализации, категорически запрещается использовать макросы явного жесткого позиционирования (такие как Rel_R, Rel_L, Rel_D и Rel_U). Вместо этого следует использовать исключительно стандартный гибкий макрос Rel(), позволяющий движку Graphviz самостоятельно и бесконфликтно рассчитывать оптимальную топологию графа на основе иерархии слоев C4.

3.2 Рекомендуемый стек локальных инструментов для реализации PlantUML

Для быстрой локальной работы, поддержки концепции Diagrams-as-Code и обеспечения конфиденциальности разрабатываемых архитектурных схем рекомендуется развернуть следующий бесплатный программный стек:

1. **Visual Studio Code:** Популярная кроссплатформенная среда разработки (IDE), используемая как единое окно для написания программного кода на Python и текстового описания архитектурных диаграмм.
2. **Расширение «PlantUML» для VS Code:** Официальный плагин, обеспечивающий полноценную подсветку синтаксиса, автодополнение элементов C4 Model и, главное, генерацию интерактивного окна предпросмотра диаграммы в реальном времени по комбинации горячих клавиш Alt+D.
3. **Локальный Docker-контейнер PlantUML Server:** Рекомендуемый сервер отрисовки. Позволяет генерировать финальные изображения локально на компьютере, не отправляя исходный код архитектуры на публичные веб-серверы, что полностью исключает утечку данных.
4. **Программный пакет Graphviz:** Необходимый системный движок визуализации графов. Он выполняет математический расчет координат размещения блоков и минимизирует пересечения стрелок на сложных иерархических диаграммах уровней L2 и L3.

Примечание: в индустрии также широко распространены альтернативные инструменты описания архитектуры кодом, такие как специализированная экосистема Structurizr (включающая язык Structurizr DSL и локальный визуализатор Structurizr Lite), реализующая принцип «единого источника правды» (Single Source of Truth), и легковесный инструмент Mermaid.js, предназначенный для нативной интеграции схем непосредственно в Markdown-документацию репозиториев (GitHub/GitLab) и базы знаний (Notion/Confluence). Однако, в рамках данного учебного курса указанные инструменты подробно не рассматриваются, и все практические задания выполняются строго с использованием экосистемы PlantUML.

4 ИНТЕГРАЦИЯ И СЕГМЕНТАЦИЯ ПРОЕКТА В КОМАНДЕ: как сегментировать проект в команде в контексте C4

4.1 Общие положения и дуальный подход к обучению

С целью имитации реальных индустриальных условий разработки программного обеспечения и одновременного обеспечения полноценного персонального контроля знаний, в рамках курса применяется гибридный командно-индивидуальный подход:

- Командная работа: Студенты объединяются в группы по 5 человек (1 Тимлид и 4 Разработчика/ Аналитика) для создания единой, бесшовно интегрированной ИТ-экосистемы. Роль Тимлида заключается в сквозной координации всех уровней абстракции C4 Model, отслеживании архитектурных границ и контроле целостности интеграционных интерфейсов.
- Индивидуальная работа (Защита ЛР): Каждый студент обязан пройти полноценный личный контроль знаний, самостоятельно и локально разработав и защитив все 5 уровней абстракции архитектуры (L0, L1, L2, L3, L4, динамика, развертывание) применительно к закрепленной за ним зоне ответственности.

4.2 Детальное распределение обязанностей по лабораторным работам

Для обеспечения прозрачности оценки ниже приведена жесткая матрица зон ответственности Лидера команды и Разработчиков на каждом этапе проектирования**:**

Лабораторная работа №1

(System Context - L1)

- Обязанности Тимлида: Проектирует общую глобальную диаграмму контекста всей системы. Очерчивает внешние границы, роли водителей, операторов и сторонние API. Проводит интеграционное ревью.
- Обязанности Разработчика (Студенты 1-4): Разрабатывает индивидуальную версию диаграммы L1, детально фокусируясь на окружении своего контейнера (например, Студент 1 отображает все внешние точки входа для мобильного приложения). Обязательно сдает и защищает ЛР №1 лично.

(Containers - L2)

- Обязанности Тимлида: Определяет глобальный технологический стек, структуру СУБД, правила распределения микросервисов. Координирует интерфейсы и протоколы обмена (HTTPS, MQTT, SQL).

- Обязанности Разработчика (Студенты 1-4): Проектирует схему контейнеров с фокусом на свой выделенный узел, детально аргументируя выбранный технологический стек (React Native, FastAPI, Mosquitto и т.д.). Лично защищает ЛР №2.

Лабораторная работа №2 (Components - L3)

- Обязанности Тимлида: Сам схему компонентов не пишет, но может объяснить каждую схему команды, защищает схему по выбору преподавателя. Контролирует совместимость ID стыковочных модулей (`auth_comp`, `booking_comp`, `payment_comp`) бэкенда с фронтендом и IoT.
- Обязанности Разработчика (Студенты 1-4): Полностью изолированно проводит глубокую декомпозицию своего контейнера на уровне внутренних модулей, описывает внутрипроцессные вызовы (`Python Call`). Защищает ЛР №3.

Лабораторная работа №3 (Code UML & Python - L4)

- Обязанности Тимлида: Выступает в роли Главного Архитектора (Enterprise Architect). Код не пишет, но знает любой код команды по выбору преподавателя. Проводит сквозное Code Review в Pull Requests. Контролирует сопряжение сигнатур методов и строгое соблюдение принципов SOLID и Type Hinting на стыках.
- Обязанности Разработчика (Студенты 1-4): Выбирает ключевой внутренний компонент своего L3-модуля и детализирует его до уровня UML Class Diagram. Пишет чистый ООП-код на Python с обязательной практической реализацией паттерна «Стратегия» (Strategy) для динамической подмены бизнес-логики. Лично сдает работающий код и диаграмму классов.

Лабораторная работа №4 (Landscape L0 & Dynamic Diagram& Deployment Diagram)

- Обязанности Тимлида: 1. Часть Ландшафт (L0): Проектирует глобальную интеграцию системы в экосистему Smart City (Enterprise Boundary), связывая ее с CRM и IAM.
  2. Часть Динамика: Собирает глобальный файл `dynamic_process.puml`, объединяющий вызовы от всех участников. 3. Развертывание: Собирает общую схему инфраструктуры развертывания проекта, объединяя узлы хостинга и контролируя политики безопасности.
- Обязанности Разработчика (Студенты 1-4): 1. Часть Ландшафт (L0): Локально проектирует индивидуальную схему макроуровня L0, показывая как его конкретный модуль встраивается в инфраструктуру города.
  2. Часть Динамика: Передает Тимлиду точные названия функций и протоколы взаимодействия своего контейнера для общей сборки. Самостоятельно строит и защищает хронологический runtime-сценарий. 3. Развертывание: Проектирует локальный сегмент диаграммы развертывания для своего контейнера, указывает используемые порты, протоколы взаимодействия и топологию распределения.

Лабораторная работа №5 (Интеграция, сборка проекта)

- Обязанности Тимлида: Выступает в роли DevOps-инженера и Релиз-инженера проекта. Настраивает и размещает в репозитории конфигурационный файл автоматического CI/CD

пайплайна (.github/workflows/render-docs.yml). Инициализирует репозиторий, прописывает правила защиты главной ветки (Branch Protection Rules для main), проверяет поступающие от разработчиков Pull Requests. Проводит сквозное архитектурное и техническое Code Review, выявляет нарушения стандартов кодирования и глоссария ID. Выполняет слияние одобренных веток (Merge) и отвечает за корректность генерации итоговых растровых артефактов (PNG/SVG) в базу знаний проекта.

- Обязанности Разработчика (Студенты 1-4): Самостоятельно разворачивает и конфигурирует локальный стек инструментов (VS Code, официальный плагин PlantUML, Docker-контейнер PlantUML Server для безопасного локального рендеринга). Полностью переводит свои локальные проектные наработки уровней L2 (Контейнеры) и L3 (Компоненты) в строгий текстовый формат puml-кода. Обеспечивает точное соответствие связей единому глоссарию глобальных идентификаторов (Glossary IDs). Создает изолированную feature-ветку в Git и открывает Pull Request в ветку main. В случае падения автоматической сборки с индикатором «Красный крест» (✕) самостоятельно анализирует логи терминала CI/CD, исправляет синтаксические ошибки и успешно проходит процедуру защиты кода перед Тимлидом и преподавателем.

СТРОГОЕ ПРАВИЛО ИНДИВИДУАЛЬНОЙ ЗАЩИТЫ: *Несмотря на то, что Тимлид отвечает за финальную интеграцию мастер-файлов (main.puml) в Git-репозитории, КАЖДЫЙ студент для успешного прохождения аттестации обязан самостоятельно спроектировать ЛОКАЛЬНЫЕ версии диаграмм Level 0 (Ландшафт) и Level 1 (Контекст). На защите лабораторных работ преподаватель проводит индивидуальный опрос по всем 5 уровням модели C4.*

4.3 Организация Git-репозитория и автоматическая сборка

Для исключения конфликтов при слиянии веток, проект организуется по модульному принципу. Архитектура описывается в виде кода (Diagrams-as-Code) с помощью PlantUML.

Структура папок проекта

```
smart-parking-docs/
├── .github/workflows/
│   └── render-docs.yml      # CI/CD сценарий авто-сборки схем
├── src/
│   ├── main.puml                # Главный файл сборщика (ведет Тимлид)
│   └── containers/              # Компоненты L3 (ведут студенты)
│       ├── student1-mobile.puml
│       ├── student2-admin.puml
│       ├── student3-backend.puml
│       └── student4-iot.puml
└── README.md                    # Описание и регламент команды
```

Автоматический контроль синтаксиса (CI/CD)

В репозитории настраивается автоматическая сборка (GitHub Actions или GitLab CI). При каждом Push или Pull Request система автоматически скачивает PlantUML, проверяет синтаксис файлов и генерирует готовые графические схемы. Если студент совершил синтаксическую ошибку — сборка выдает статус «Красный крест» (✕), и такая работа автоматически отклоняется от проверки.

4.4 Единые стандарты кодирования и глоссарий ID

Связи в PlantUML строятся по строгим текстовым идентификаторам элементов (ID). Все студенты обязаны использовать только зафиксированные глобальные ID:

- driver — Водитель (Пользователь)
- admin — Оператор парковки (Пользователь)
- mobile_app — Мобильное приложение водителя (Контейнер)
- admin_web — Веб-панель администратора (Контейнер)
- api_gateway — Бэкенд / API Gateway (Контейнер)
- database — Центральная база данных PostgreSQL (БД)
- iot_service — Сервис интеграции с IoT-оборудованием (Контейнер)
- payment_gateway — Внешний платежный шлюз (Внешняя система)
- iot_sensors — Датчики и шлагбаумы на парковке (Внешняя система)

Обязательные стыковочные компоненты Бэкенда (Студент 3)

Внутри своего файла Студент 3 обязан выделить компоненты со следующими ID для подключения других студентов:

- auth_comp — Модуль авторизации (к нему подключаются Студент 1 и 2)
- booking_comp — Модуль бронирования мест (к нему подключаются Студент 1 и Студент 4)
- payment_comp — Модуль оплаты (к нему подключается Студент 1)
-

Правила оформления диаграмм (Запреты)

- **Запрет жесткого позиционирования:** Категорически запрещено использовать стрелки с указанием направления (Rel_D, Rel_R, Rel_L). Разрешен только макрос Rel(от, кому, 'описание', 'протокол'). Направление определяется движком PlantUML автоматически.
- **Запрет локальной стилизации:** Любые кастомные цвета, шрифты и стили оформления (skinparam) выносятся в главный файл. Файлы студентов содержат исключительно чистую логику структуры.
- **Обязательность протоколов:** Каждая стрелка связи должна содержать понятное описание бизнес-действия и технический протокол передачи данных (например, HTTPS/JSON, gRPC, SQL/TCP 5432).

4.5. Контроль выполнения со стороны преподавателя

Преподаватель осуществляет верхнеуровневый аудит процессов на основе индустриальных метрик в Git:

- **Аудит активности участников:** Оценивается распределение коммитов по времени и авторам во вкладке репозитория (Contributors/Analytics). Если весь код залит одним числом от лица Тимлида — команда штрафуется за отсутствие командного взаимодействия.
- **Проверка Pull Requests:** Преподаватель оценивает качество ведения проектного диалога. Тимлид должен оставлять комментарии в коде студентов при проведении Code Review до слияния веток.
- **Защита сквозной архитектуры:** На итоговую защиту выносится скомпилированная CI/CD-системой общая интерактивная карта архитектуры, где преподаватель может оценить связность истории и детализацию каждого слоя.

5. ПРИМЕР выполнения лабораторных работ

Лабораторная работа №1. Диаграмма системного контекста (Level 1)

В качестве примера рассматривается проект автоматизированной системы

Первый уровень абстракции C4 Model — это Диаграмма системного контекста.

Она позволяет взглянуть на проектируемую систему с «высоты птичьего полета». Система рассматривается как единый неделимый «черный ящик» (Black Box).

Основная цель диаграммы — очертить жесткие внешние границы проектируемой системы и показать ее взаимодействие с внешним миром.

Ключевые элементы диаграммы:

- Персоны (User Person),
- Проектируемая программная система (Software System),
- Внешние системы (External Systems), Связи (Relationships).

Между элементами диаграммы устанавливаются связи (стрелки), указывающие инициатора запроса и направление движения данных.

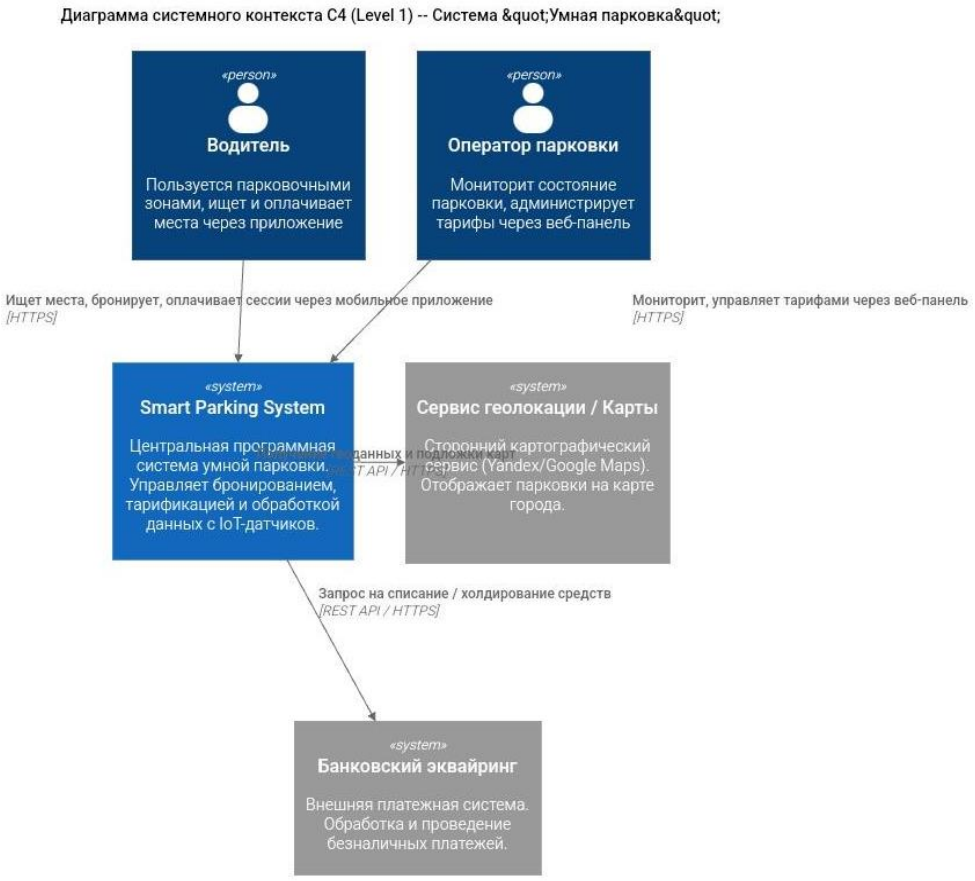


Рисунок 1 Диаграмма системного контекста.

Текстовая спецификация элементов и связей диаграммы системного контекста L1:

- Блоки элементов: отображают ключевых акторов и системы. Синие блоки со скруглениями сверху (Person) представляют пользователей. Внутри блока «Водитель» указано его служебное назначение — поиск и оплата мест; внутри блока «Оператор парковки» — администрирование тарифов и мониторинг. Центральный темно-синий прямоугольник (System) — «Smart Parking System», проектируемое монолитное ядро. Серые прямоугольники (System_Ext) — внешние независимые системы: «Сервис геолокации / Карты» и «Банковский эквайринг». Любые элементы могут связываться между собой, если между ними существует информационный поток.
- Стрелки (Связи взаимодействия): строго направленные линии, указывающие инициатора запроса и направление движения данных. Между двумя конкретными элементами может существовать одна или несколько стрелок, если они выполняют концептуально разные бизнес-задачи. Стрелка от «Водителя» к центральной системе означает отправку команд бронирования и оплаты по протоколу HTTPS. Стрелка от центральной системы к «Банковскому эквайрингу» направлена в сторону внешней системы, описывая запрос на списание денежных средств по протоколу REST API / HTTPS. Связь со структурой карт (geolocation) направлена вправо для оптимизации визуального макета.

Второй уровень абстракции — Диаграмма контейнеров. Контейнер в терминологии C4 Model — это не Docker-контейнер, а обособленная, независимо развертываемая исполняемая сущность, подсистема или хранилище данных, выполняющееся в собственном адресном пространстве.

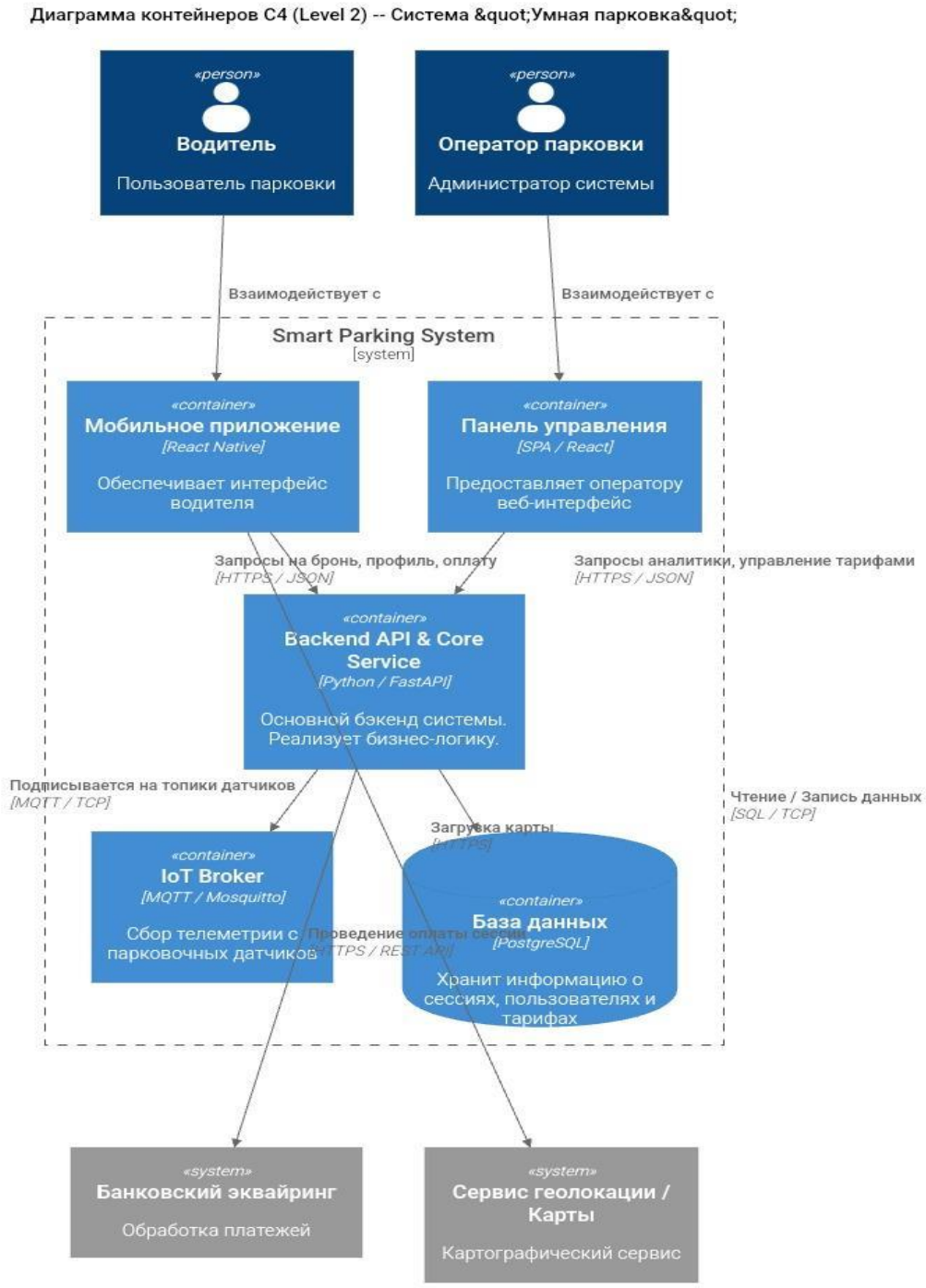


Рисунок 2. Диаграмма контейнеров.

```
@startuml "Smart Parking - Global Container View"
' Подключение официальной библиотеки макросов C4 напрямую из GitHub
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml

LAYOUT_WITH_LEGEND()
title Сводная диаграмма контейнеров ИС "Умная парковка"

Person(driver, "Водитель", "Пользуется парковочными зонами, ищет и оплачивает места")
Person(admin, "Оператор парковки", "Мониторит состояние парковки, администрирует тарифы")
System_Ext(payment_gateway, "Банковский эквайринг", "Внешняя платежная система")

System_Boundary(smart_parking, "Smart Parking System") {
    Container(mobile_app, "Мобильное приложение водителя", "React Native", "Интерфейс водителя для поиска парковок и оплаты.")
    Container(admin_web, "Панель администратора", "React / TypeScript", "Интерфейс оператора для мониторинга.")
    Container(api_gateway, "API Gateway & Backend", "Python / FastAPI", "Маршрутизирует запросы и управляет бизнес-логикой.")
    ContainerDb(database, "Реляционная БД", "PostgreSQL", "Хранит данные пользователей и статусы мест.")
    Container(iot_controller, "Контроллер парковки", "Python / Asyncio (IoT Сервис)", "Сбор телеметрии с датчиков.")
}

Rel(driver, mobile_app, "Взаимодействует с", "HTTPS")
Rel(admin, admin_web, "Взаимодействует с", "HTTPS")
Rel(mobile_app, api_gateway, "Запросы на бронь", "HTTPS/JSON")
Rel(admin_web, api_gateway, "Запросы аналитики", "HTTPS/JSON")
Rel(api_gateway, database, "Чтение / Запись", "SQL")
Rel(api_gateway, payment_gateway, "Проведение оплаты", "HTTPS/REST")
@enduml
```

Текстовая спецификация элементов и связей диаграммы контейнеров L2:

- Контейнеры внутри границы системы: Пунктирная рамка System_Boundary объединяет 5 внутренних контейнеров:
  - «Мобильное приложение» (React Native) и «Панель управления» (SPA / React) — это клиентские приложения.
  - «Backend API & Core Service» (Python / FastAPI) — исполняемый контейнер бизнес-логики.

- «IoT Broker» (MQTT / Mosquitto) — брокер сообщений для сбора телеметрии.
- «База данных» (PostgreSQL) — контейнер хранения данных.
  Каждый контейнер имеет явно зафиксированный технологический стек.

• Информационные потоки и сетевые протоколы:

Стрелки детально описывают техническое межсервисное взаимодействие^

- Мобильное приложение и веб-панель отправляют запросы к единому бэкенду по протоколу HTTPS / JSON.
- Бэкенд взаимодействует с базой данных по сетевому протоколу SQL / TCP (чтение/запись данных).
- Взаимодействие бэкенда с IoT-брокером идет по протоколу MQTT / TCP (подписка на топики датчиков).
  Направленность стрелок отражает, какая сущность является инициатором сетевого сеанса.

Лабораторная работа №2. Диаграмма компонентов (Level 3)

Третий уровень абстракции — Диаграмма компонентов — осуществляет декомпозицию одного конкретного контейнера на внутренние логические блоки: компоненты. Согласно определению C4, компонент — это совокупность связанных между собой классов или модулей, объединенных общим интерфейсом, выполняющихся внутри единого процесса (контейнера) и не развертываемых отдельно.

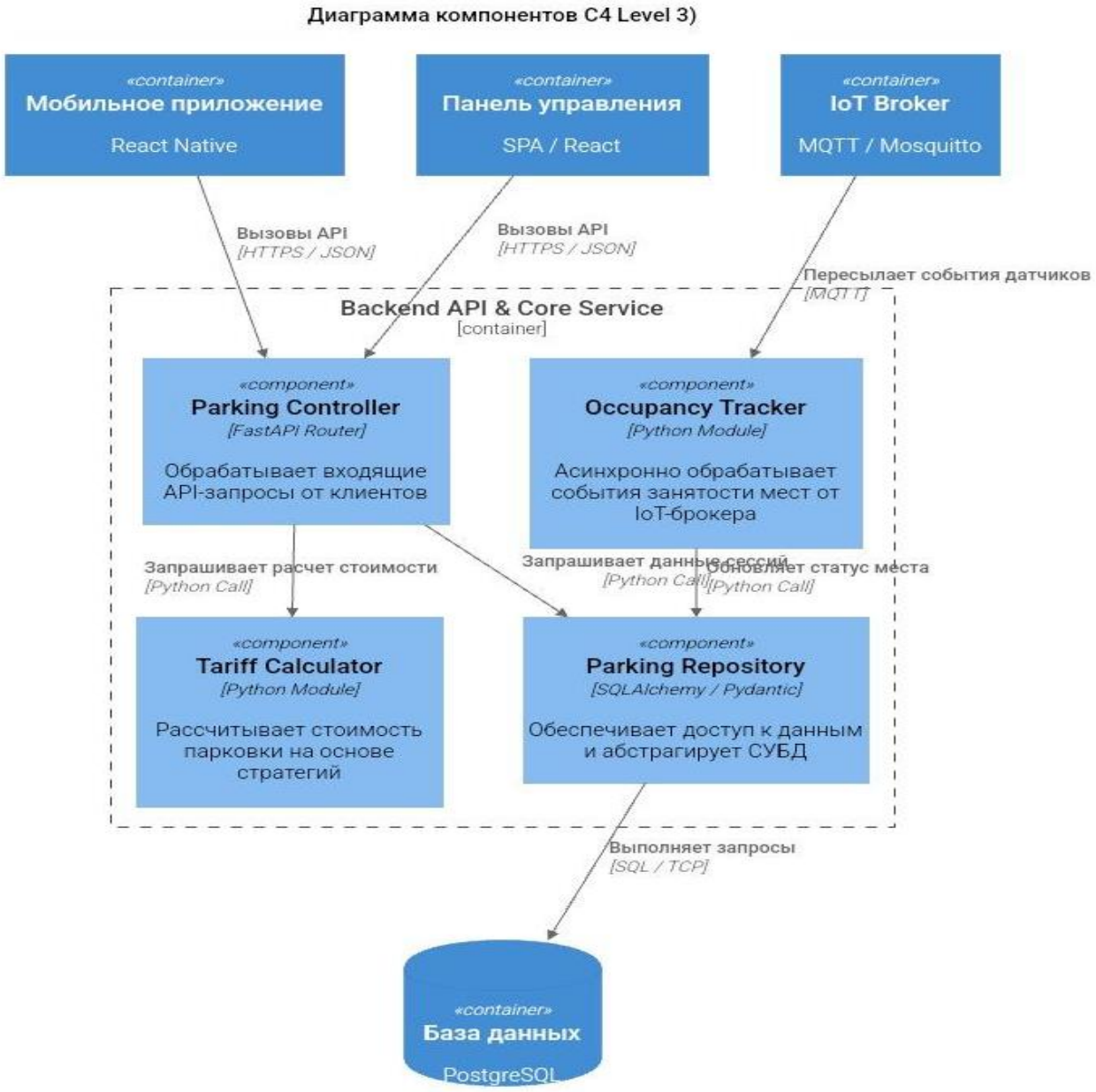


Рисунок 3. Диаграмма компонентов бэкенда.

Текстовая спецификация элементов и связей диаграммы компонентов L3:

- Внутренние компоненты бэкенда: внутри контейнерной границы Container_Boundary выделено 4 компонента

  - «Parking Controller» (FastAPI Router) отвечает за маршрутизацию внешних запросов.
  - «Occupancy Tracker» (асинхронный модуль Python) слушает телеметрию от брокера датчиков.
  - «Tariff Calculator» (изолированный модуль бизнес-логики) инкапсулирует правила расчета цен.
  - «Parking Repository» (слой доступа к данным на SQLAlchemy) абстрагирует SQL-запросы к СУБД.
    Внешние по отношению к бэкенду контейнеры (база данных, приложения, брокер) вынесены за рамки бэкенд-границы.

- Внутрипроцессное взаимодействие: Стрелки между внутренними компонентами представляют собой не сетевые протоколы, а прямые программные вызовы (Python Call) в общем адресном пространстве памяти.

Контроллер вызывает калькулятор тарифов для расчета стоимости и репозиторий для получения сессий.

Трекер занятости мест также вызывает репозиторий для сохранения статуса места.
Единственная сетевая стрелка изнутри компонента наружу ведет от репозитория к контейнеру базы данных по протоколу SQL / TCP.

Лабораторная работа №3. Диаграмма кода (Level 4)

Четвертый уровень абстракции — Диаграмма кода. Этот уровень показывает, как конкретный архитектурный компонент реализуется в кодовой базе в виде классов, интерфейсов и их объектно-ориентированных связей. Использование паттерна «Стратегия» (Strategy) позволяет динамически подменять алгоритмы тарификации (почасовой, ночной, праздничный) без изменения основного кода контекста, соблюдая принцип SOLID Open/Closed.

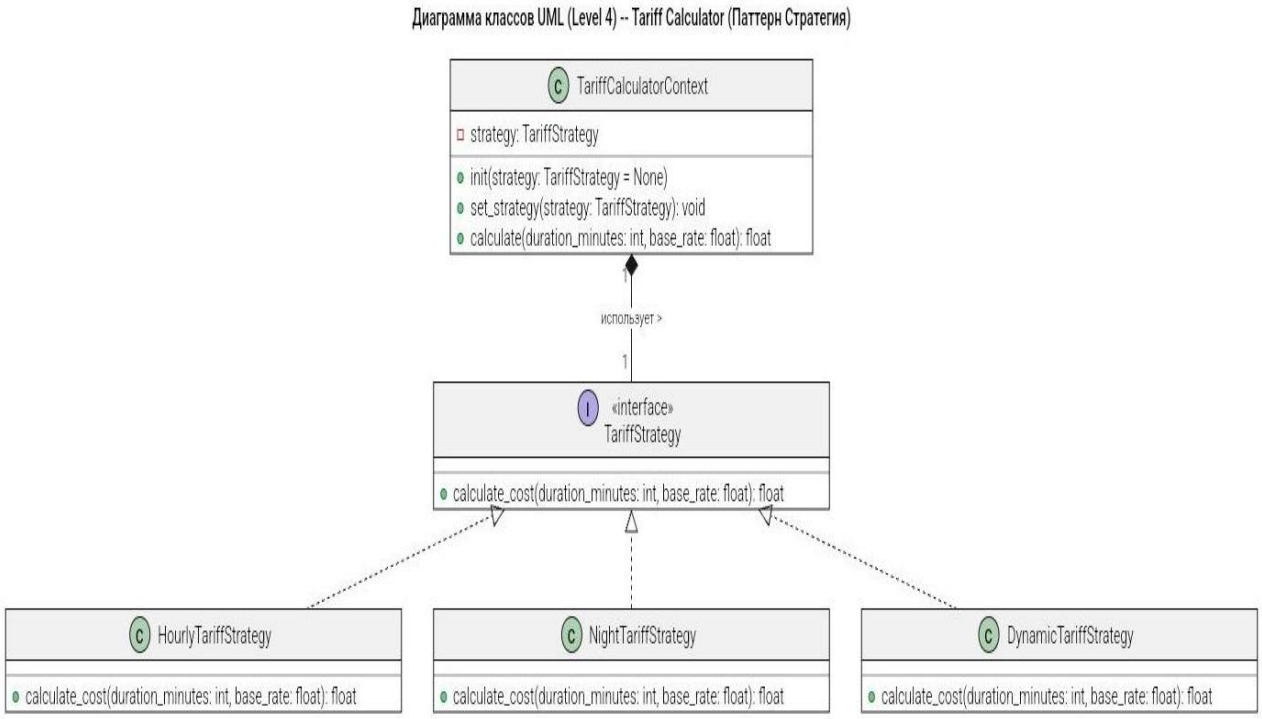


Рисунок 4. Код UML Class Diagram для компонента Tariff Calculator:

Текстовая спецификация элементов и связей диаграммы классов L4:

- Классы и интерфейсы: Отображаются в виде специализированных таблиц-карточек UML. Основной управляющий класс — «TariffCalculatorContext», содержащий приватное поле ссылки на стратегию и методы ее смены. Элемент со стереотипом «interface» — «TariffStrategy», фиксирующий единый контракт метода расчета стоимости. Карточки классов «HourlyTariffStrategy», «NightTariffStrategy» и «DynamicTariffStrategy» представляют собой конкретные алгоритмические реализации расчета цены парковки.

- Типы объектно-ориентированных связей: Стрелочки соответствуют каноническому синтаксису классов UML. Линия с ромбом обозначает жесткую композицию: контекст содержит внутри себя ровно один экземпляр интерфейса стратегии (отношение 1 к 1). Стрелки с незакрашенным треугольником и пунктирной линией обозначают реализацию интерфейса конкретными классами стратегий, что демонстрирует выполнение принципа Open/Closed из SOLID (подмена алгоритмов без изменения контекста).

Лабораторная работа №4. Макроархитектура, динамические процессы и инфраструктура развертывания.

Лабораторная работа №4 объединяет два специализированных подтипа представлений в C4 Model, которые выходят за рамки классического пошагового «зумирования» изолированного приложения, — Диаграмму системного ландшафта (Level 0) и Динамическую диаграмму (Dynamic Diagram).

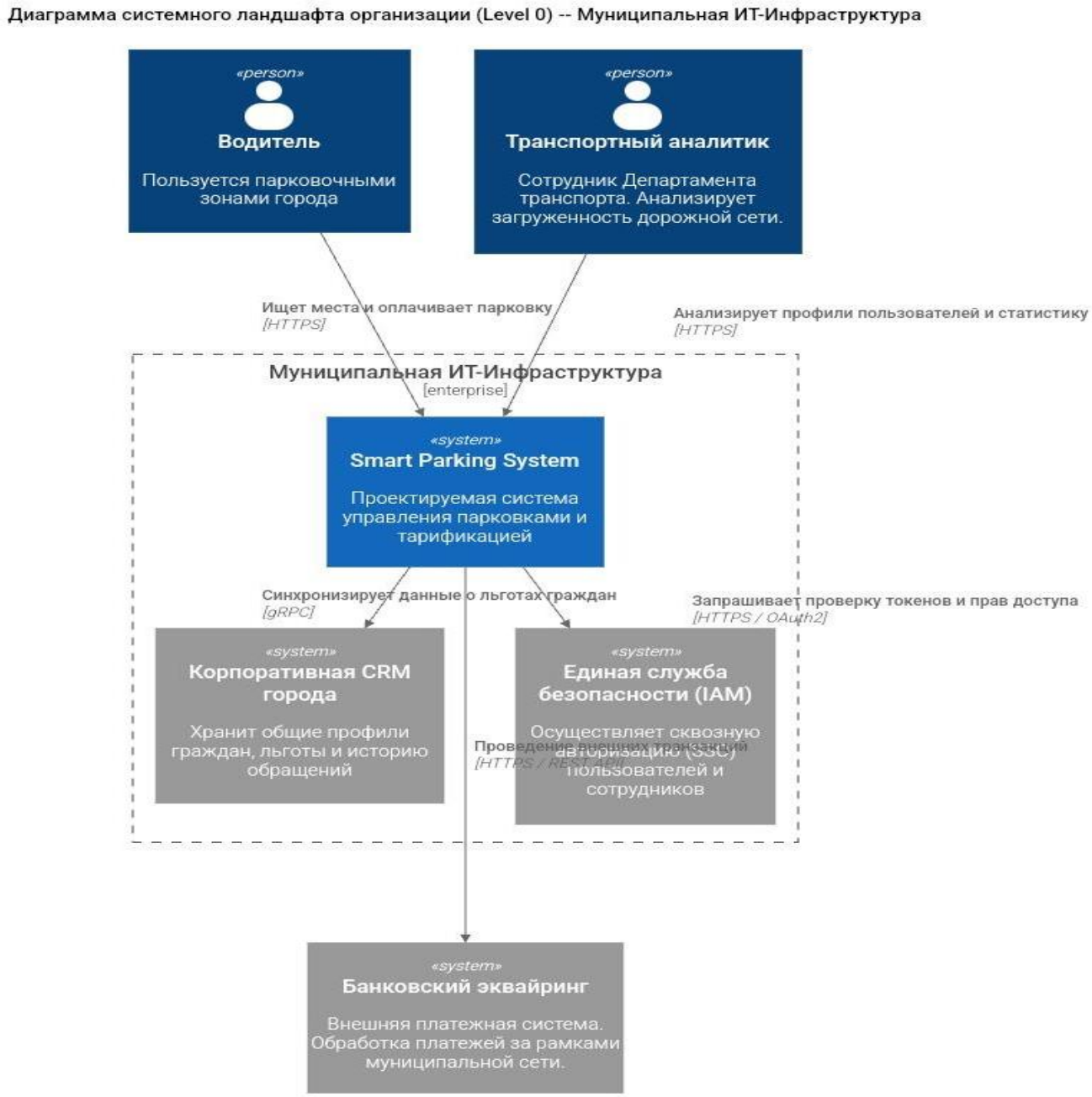


Рисунок 5. Диаграмма системного ландшафта (Level 0).

Текстовая спецификация элементов и связей ландшафтной диаграмм:

- Элементы макроуровня (Level 0): Граница Enterprise_Boundary («Муниципальная ИТ-Инфраструктура») очерчивает периметр города.
- Внутри нее находятся блоки нашей системы «Smart Parking System» и смежных городских ИТ-платформ: «Корпоративная CRM города» и «Единая служба безопасности (IAM)».
- Блок «Банковский эквайринг» находится за пределами этой муниципальной границы.
- Стрелки описывают защищенные межсистемные интеграции внутри закрытого контура организации: gRPC для синхронизации льгот с CRM и HTTPS / OAuth2 для проверки сквозной авторизации в IAM.

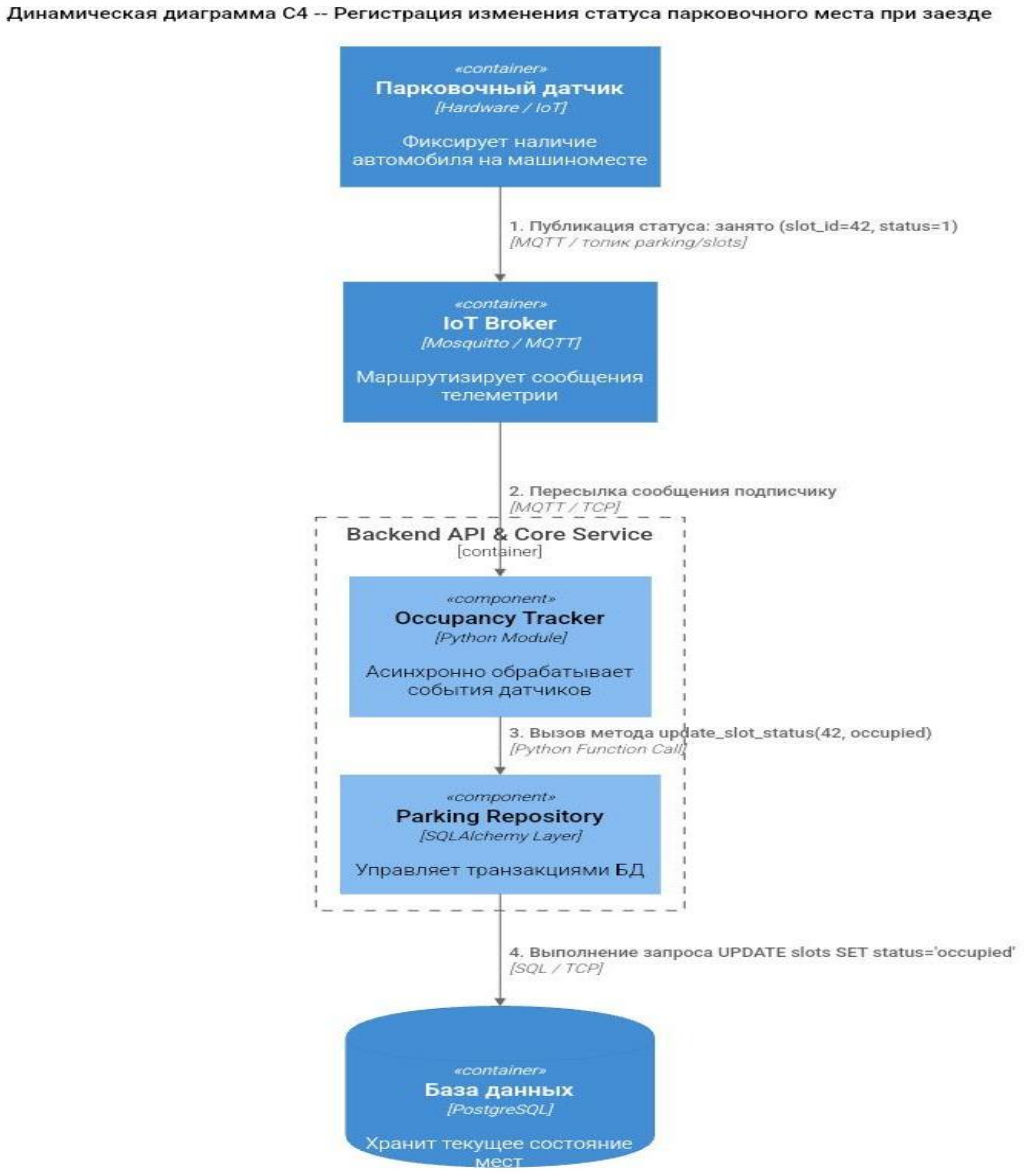


Рисунок 6. Динамическая диаграммы (Рантайм-процесс изменения статуса места).

```
@startuml "Smart Parking - Dynamic Diagram"
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml

title Динамическая диаграмма — Регистрация изменения статуса парковочного места

Container(sensor, "Парковочный датчик", "Hardware / IoT", "Фиксирует наличие автомобиля")
```

```
Container(broker, "IoT Broker", "Mosquitto / MQTT", "Маршрутизирует сообщения телеметрии")

System_Boundary(backend, "Backend API & Core Service") {
   Component(tracker, "Occupancy Tracker", "Python Module", "Асинхронно обрабатывает события датчиков")
   Component(repo, "Parking Repository", "SQLAlchemy Layer", "Управляет транзакциями БД")
}
ContainerDb(database, "База данных", "PostgreSQL", "Хранит текущее состояние мест")

Rel(sensor, broker, "1. Публикация статуса: занято", "MQTT / топик parking/slots")
Rel(broker, tracker, "2. Пересылка сообщения подписчику", "MQTT / TCP")
Rel(tracker, repo, "3. Вызов метода update_slot_status", "Python Function Call")
Rel(repo, database, "4. Выполнение запроса UPDATE", "SQL / TCP")
@enduml
```

Текстовая спецификация элементов и связей динамической диаграммы:

Потоки динамической диаграммы:

В отличие от статических схем, стрелочки на динамической диаграмме имеют строгую последовательную нумерацию (от 1 до 4), фиксирующую хронологию сквозного рантайм-процесса:

- Стрелка 1: аппаратный «Парковочный датчик» отправляет сообщение в «IoT Broker» по протоколу MQTT в топик парковки.
- Стрелка 2: брокер пересылает данные асинхронному компоненту «Occupancy Tracker».
- Стрелка 3: трекер выполняет внутренний вызов метода репозитория.
- Стрелка 4: репозиторий отправляет SQL-запрос UPDATE в контейнер базы данных PostgreSQL для фиксации занятости машиноместа.

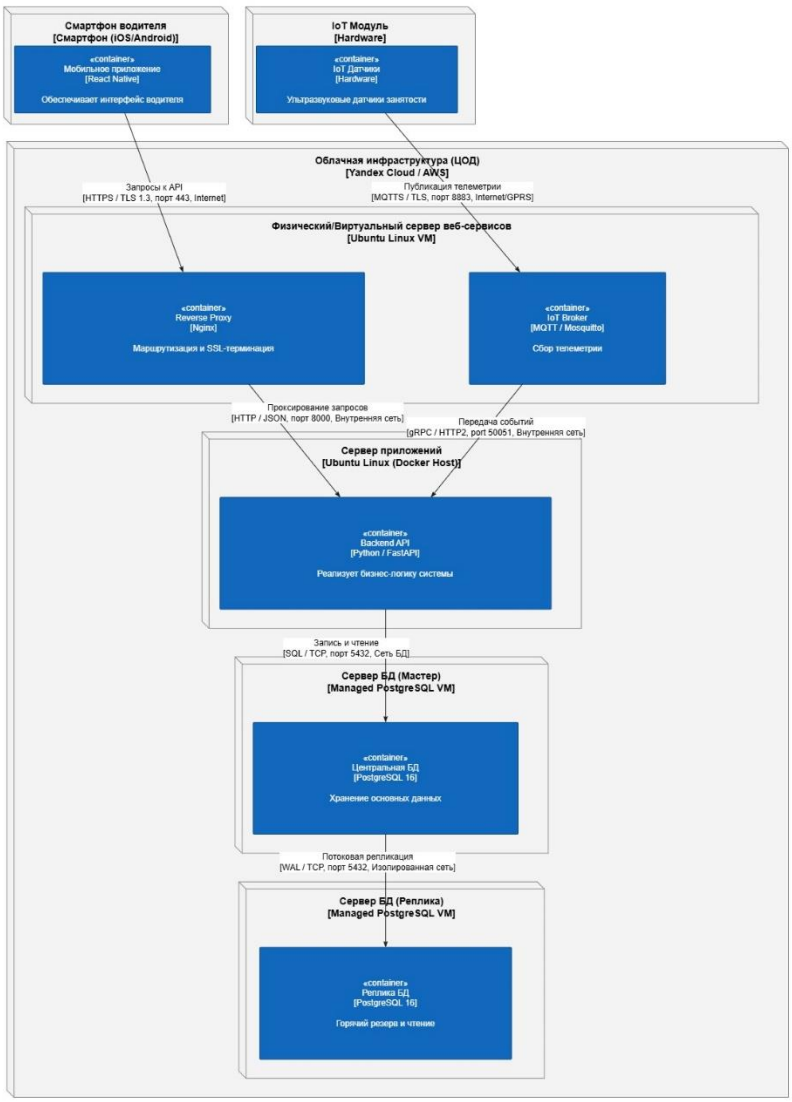


Рисунок 7. Диаграмма развертывания. (L2)

```
@startuml "Smart Parking - Deployment Diagram"
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Deployment.puml

title Диаграмма развертывания системы «Умная парковка»

Deployment_Node(mob, "Смартфон водителя", "Smart Device") {
    Container(mobile, "Мобильное приложение", "React Native")
}

Deployment_Node(cloud, "Облачная инфраструктура (ЦОД)", "Yandex Cloud / AWS") {
    Deployment_Node(web_server, "Физический/Виртуальный сервер веб-сервисов", "Ubuntu Linux VM") {
```

```
        Container(nginx, "Reverse Proxy", "Nginx")
        Container(mosquitto, "IoT Broker", "Mosquitto / MQTT")
    }
    Deployment_Node(app_server, "Сервер приложений", "Ubuntu (Docker Host)") {
        Container(backend, "Backend API", "Python / FastAPI")
    }
    Deployment_Node(db_master, "Сервер БД (Мастер)", "Managed PostgreSQL VM") {
        ContainerDb(db, "Центральная БД", "PostgreSQL 16")
    }
}

Rel(mobile, nginx, "Запросы к API", "HTTPS / порт 443")
Rel(nginx, backend, "Проксирование запросов", "HTTP / JSON, порт 8000")
Rel(backend, db, "Запись и чтение", "SQL / TCP, порт 5432")
@enduml
```

1. Спецификация элементов инфраструктуры (Deployment Nodes & Containers)

1.1. Сторона клиента и периферии (Edge & Client Nodes)

- Узел: Смартфон водителя
  - *Тип узла:* Мобильное устройство пользователя (под управлением операционных систем iOS или Android).
  - *Размещенный контейнер:*
    - Мобильное приложение [React Native]: Обеспечивает графический интерфейс водителя для отображения карты, статусов и отправки запросов в API.
- Узел: IoT Модуль
  - *Тип узла:* Физическое микроконтроллерное устройство (Hardware), установленное на парковочном объекте.
  - *Размещенный контейнер:*
    - IoT Датчики [Hardware]: Ультразвуковые датчики занятости. Фиксируют физическое присутствие автомобиля и генерируют сырые события.

1.2. Облачная инфраструктура (Cloud Infrastructure)

- Группа узлов / Окружение: Облачная инфраструктура (ЦОД)
  - *Провайдер:* Виртуальное частное облако (Yandex Cloud / AWS VPC).
- Узел: Физический/Виртуальный сервер веб-сервисов [Ubuntu Linux VM]
  - *Характеристики:* Виртуальная машина с публичным IP-адресом, выполняющая роль фронтального шлюза.
  - *Размещенные контейнеры:*
    - Reverse Proxy [Nginx]: Отвечает за маршрутизацию и SSL-терминацию входящих HTTP-запросов от пользователей.
    - IoT Broker [MQTT / Mosquitto]: Брокер сообщений, предназначенный для сбора легковесной телеметрии с периферийных IoT-датчиков.
- Узел: Сервер приложений [Ubuntu Linux (Docker Host)]

    - *Характеристики:* Изолированная виртуальная машина в приватной подсети, предназначенная для работы Docker-контейнеров приложения.
    - *Размещенный контейнер:*
        - Backend API [Python / FastAPI]: Реализует бизнес-логику системы, валидирует данные, обрабатывает запросы от Nginx и события от Mosquitto.
- Узел: Сервер БД (Мастер) [Managed PostgreSQL VM]
    - *Характеристики:* Высокопроизводительная виртуальная машина (или управляемый облачный сервис СУБД) с быстрыми накопителями.
    - *Размещенный контейнер:*
        - Центральная БД [PostgreSQL 16]: Хранение основных данных. Принимает все транзакции на запись и чтение.
- Узел: Сервер БД (Реплика) [Managed PostgreSQL VM]
    - *Характеристики:* Ведомая виртуальная машина, развернутая для обеспечения высокой доступности данных.
    - *Размещенный контейнер:*
        - Реплика БД [PostgreSQL 16]: Горячий резерв и чтение данных. Используется для резервного копирования и разгрузки мастера от тяжелых выборок.

2. Спецификация связей (Сетевые интерфейсы и протоколы)

Исходный элемент	Целевой элемент	Описание взаимодействия	Протокол и Порт	Сетевая среда
Мобильное приложение	Reverse Proxy (Nginx)	Запросы к API	HTTPS / TLS 1.3, порт 443	Открытая сеть (Internet)
IoT Датчики	IoT Broker (Mosquitto)	Публикация телеметрии	MQTTS / TLS, порт 8883	Интернет / GPRS
Reverse Proxy (Nginx)	Backend API	Проксирование запросов	HTTP / JSON, порт 8000	Внутренняя сеть ЦОД
IoT Broker (Mosquitto)	Backend API	Передача событий	gRPC / HTTP2, порт 50051	Внутренняя сеть ЦОД
Backend API	Центральная БД (Master)	Запись и чтение	SQL / TCP, порт 5432	Сеть БД (Выделенная подсеть)
Центральная БД (Master)	Реплика БД (Standby)	Потоковая репликация	WAL / TCP, порт 5432	Изолированная сеть репликации

3. Политика безопасности периметра

    1. Публичный контур: Прямой доступ из глобальной сети Интернет разрешен только к портам 443 (Nginx) и 8883 (Mosquitto) на сервере веб-сервисов.

2. Приватный контур: Сервер приложений и серверы баз данных полностью закрыты от внешнего мира межсетевыми экранами. Доступ к ним возможен только по внутренним сетевым интерфейсам из доверенных сегментов ЦОД.

Лабораторная работа №5. C4 Model. Интеграция, сборка проекта.

Финальный этап проектирования системы «Умная парковка» заключается в переводе разработанных PlantUML-моделей в автоматизированный цикл CI/CD. В данном примере демонстрируется настройка конфигурационного файла пайплайна и процесс интеграции локальных диаграмм разработчиков в мастер-ветку репозитория.

Ниже представлен практический пример выполнения Лабораторной работы №5 для сквозного проекта «Умная парковка».

1. Настройка конфигурации GitHub Actions (.github/workflows/render-docs.yml)

Для автоматической валидации синтаксиса и рендеринга диаграмм C4 в репозитории настраивается следующий декларативный пайплайн CI/CD:

```
name: Auto Render C4 Architecture Docs
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  validate-and-render:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository code
        uses: actions/checkout@v3
      - name: Setup Java Environment
        uses: actions/setup-java@v3
        with:
          distribution: 'temurin'
          java-version: '11'
      - name: Install Graphviz Engine
        run: |
          sudo apt-get update
          sudo apt-get install -y graphviz
```

```
      - name: Download PlantUML Jar Executable
        run: |
          wget https://github.com/plantuml/plantuml/releases/download/v1.2023.10/plantuml-1.2023.10.jar -O plantuml.jar
      - name: Validate Syntax and Compile Diagrams to PNG
        run: |
          java -jar plantuml.jar -verbose -png -o "../docs/images" "src/**/*.puml"
      - name: Commit and Push Rendered Artifacts
        if: github.event_name == 'push'
        run: |
          git config --global user.name 'DevOps Bot'
          git config --global user.email 'devops@stankin.ru'
          git add docs/images/*.png
          git commit -m 'docs: auto-update architecture diagrams [skip ci]' || echo 'No changes to commit'
          git push
```

2. Пример интеграционного мастер-файла сборщика (src/main.puml)

Тимлид ведет главный файл, который импортирует подсистемы, разработанные студентами, в единый контекст:

```
@startuml "Smart Parking - Global Container View"

!include core/C4_Container.puml

LAYOUT_WITH_LEGEND()

title Сводная диаграмма контейнеров ИС "Умная парковка"

' === ГЛОБАЛЬНЫЕ АКТОРЫ (Зона ответственности Тимлида) ===
Person(driver, "Водитель", "Пользуется парковочными зонами, ищет и оплачивает места")
Person(admin, "Оператор парковки", "Мониторит состояние парковки, администрирует тарифы")
System_Ext(payment_gateway, "Банковский эквайринг", "Внешняя платежная система")
```

System_Boundary(smart_parking, "Smart Parking System") {

' === СТУДЕНТ 1: Мобильное приложение ===

Container(mobile_app, "Мобильное приложение водителя", "React Native / Swift", "Позволяет искать парковки и бронировать места.")

' === СТУДЕНТ 2: Панель администратора ===

Container(admin_web, "Панель администратора", "React / TypeScript", "Интерфейс оператора для мониторинга шлагбаумов и тарифов.")

' === СТУДЕНТ 3: Серверная часть и БД ===

Container(api_gateway, "API Gateway & Backend", " Python / FastAPI", "Маршрутизирует запросы и управляет бизнес-логикой.")

ContainerDb(database, "Реляционная БД", "PostgreSQL", "Хранит данные пользователей и статусы парковочных мест.")

' === СТУДЕНТ 4: IoT-модуль ===

Container(iot_controller, "Контроллер парковки", "Python / Asyncio", "Управляет датчиками занятости мест и шлагбаумом.")

}

' === ГЛОБАЛЬНЫЕ СВЯЗИ (Совместная сборка проекта) ===

Rel(driver, mobile_app, "Взаимодействует с", "HTTPS")

Rel(admin, admin_web, "Взаимодействует с", "HTTPS")

Rel(mobile_app, api_gateway, "Запросы на бронь", "HTTPS/JSON")

Rel(admin_web, api_gateway, "Запросы аналитики", "HTTPS/JSON")

Rel(api_gateway, database, "Чтение / Запись", "SQL")

Rel(api_gateway, payment_gateway, "Проведение оплаты", "HTTPS/REST")

@enduml

3. Пример логов успешного прохождения пайплайна /

При корректном синтаксисе в терминале CI/CD отображаются следующие логи выполнения шагов:

[INFO] Checking PlantUML syntax for src/main.puml...

[INFO] Checking PlantUML syntax for src/containers/student3-backend.puml...

Parsed 2 files successfully. No structural or syntax errors detected.

Executing Graphviz layout calculation engine...

Successfully rendered artifact: docs/images/main.png [Size: 1024x768]

Successfully rendered artifact: docs/images/student3-backend.png

Job completed with exit code 0. Status: SUCCESS

4. Пример логов падения сборки при синтаксической ошибке

Если разработчик допустил ошибку (например, нарушил запрет и применил Rel_D вместо Rel, либо опечатался в ID), сборка падает:

[ERROR] Syntax Error in file: src/containers/student1-mobile.puml, Line 14

Error: Macro 'Rel_D' is banned by methodologist guidelines. Use standard 'Rel()' macro.

----> Rel_D(mobile_app, api_gateway, 'Запросы к API', 'HTTPS/JSON')

^^^^^

Error: Search code 404 - cannot compute layout matrix.

Error: process completed with exit code 1.

Pipeline failed. Status: CRITICAL FAILURE ✕